

WEB VAPT EVIDENCE FILE DOCUMENTATION

17-command-injection-id-whoami-output.png

Field	Details
Project	Enterprise Security Assessment Lab
Module	Sub-Project 1 - Web Application VAPT
Evidence Row	23 - Command Injection
Evidence File	17-command-injection-id-whoami-output.png
Result Type	OS command execution proof
Prepared For	GitHub portfolio documentation

Document Purpose

This PDF documents one specific evidence file from the Web VAPT evidence table. It embeds the actual screenshot, explains what the screenshot proves, maps it to the methodology, and records professional interpretation, risk, remediation, and redaction guidance for GitHub publication.

1. Embedded Evidence Screenshot

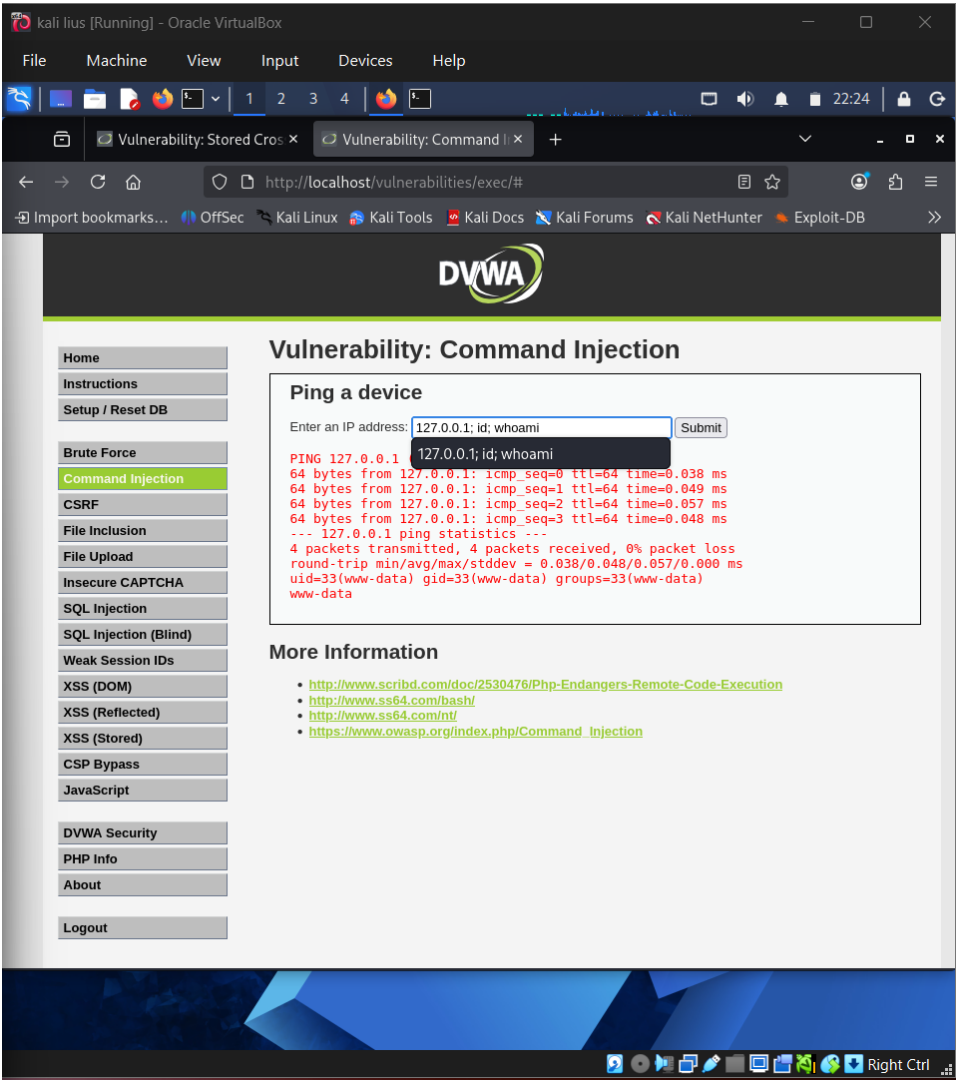


Figure 1 - Actual screenshot evidence: 17-command-injection-id-whoami-output.png

What this screenshot proves

The screenshot shows DVWA command injection output returning Unix identity details such as uid/gid and web server user context.

2. Evidence Context

Area	Value
Testing Phase	OS Command Injection Validation
Tool / Component	Browser / DVWA
Objective	Demonstrate controlled OS command execution through a vulnerable input field.
Result Type	OS command execution proof

3. Command / Workflow Used

```
127.0.0.1; id; whoami
```

The commands or workflow above represent the evidence-producing action. Placeholders such as , , or should be used in public documentation instead of live secrets or private infrastructure details.

4. Professional Interpretation

The screenshot shows DVWA command injection output returning Unix identity details such as uid/gid and web server user context.

This evidence should be treated as a supporting artifact, not as a standalone finding by itself. In a professional report, the screenshot supports a specific testing phase, validates an observation, and strengthens reproducibility when paired with methodology and remediation guidance.

5. Security Risk / Why It Matters

Command injection is critical because it may lead to arbitrary command execution, file disclosure, reverse shells, and host compromise.

For this portfolio project, the risk is described in a real-world context while clearly noting that the target was an authorized DVWA/local lab environment. This avoids exaggerating lab results as production impact.

6. Recommended Remediation / Defensive Control

Avoid shell execution, use safe APIs, apply allowlists, escape arguments safely, and run services with least privilege.

7. GitHub Redaction Requirements

Local usernames, process/user context if private, hostnames, and internal paths.

Before pushing to GitHub, confirm that screenshots do not expose cookies, session IDs, credentials, hashes, cracked passwords, private IPs, internal file paths, or any secret values. If the screenshot already contains sensitive information, blur or crop it before upload.

8. GitHub README Link Snippet

```
[Open Evidence PDF](./reports/evidence-file-documentation-with-screenshots/23_17-command-injection-id-whoami-output_evidence_documentation.pdf)
```

Legal and Ethical Note

This evidence documents authorized testing against an intentionally vulnerable local lab target. The techniques described must not be used against public or third-party systems without explicit permission.